

Exercício de Programação Orientada a Objetos

Modelagem de Moléculas e Ligações Químicas

Disciplina: Introdução à Análise de Dados

29 de abril de 2026

Contexto

Você foi contratado(a) para desenvolver um sistema que auxilie no ensino de química estrutural. O sistema deve representar **elementos químicos**, **átomos individuais** (com identidade própria dentro de uma molécula) e as **ligações** entre eles. O objetivo é praticar POO com associações, composição, encapsulamento, geração automática de identificadores e validação de regras químicas (como a regra do octeto/valência).

Requisitos Gerais

- Use **Python 3.7+**.
- Todo o código deve estar organizado em arquivos `.py` (sugestão: `elemento.py`, `atomo.py`, `ligacao.py`, `molecula.py` e um script de demonstração).
- Comente as partes importantes, use nomes significativos e trate erros básicos (ex.: validação de tipos e valores).
- Implemente todos os métodos obrigatórios descritos abaixo. Métodos extras e melhorias são bem-vindos.

Especificações das Classes

Classe Elemento

Representa um elemento químico da tabela periódica (único por símbolo). Os objetos desta classe são *imutáveis* e *hashable*, podendo ser usados como chaves de dicionário.

Atributos

- `simbolo` (str) – ex.: "H", "He", "Na", "Cl"
- `nome` (str) – ex.: "Hidrogênio", "Hélio", "Sódio", "Cloro"
- `numero_atomico` (int) – número de prótons (Z)

- `massa_atomica` (float) – em unidades de massa atômica (u)
- `valencia_maxima` (int) – número máximo de ligações covalentes que o elemento pode formar (ex.: C=4, O=2, H=1, N=3). **Obrigatório para validação.**

Métodos obrigatórios

1. `__init__(self, simbolo, nome, numero_atomico, massa_atomica, valencia_maxima)` – valide: massa e número atômico > 0; símbolo com 1 ou 2 letras, primeira maiúscula; valência entre 1 e 8.
2. `__str__(self)` – ex.: "H (Hidrogênio): Z=1, massa=1.008 u, valência=1"
3. `__hash__(self)` – retorne `hash(self.simbolo)`.
4. `__eq__(self, other)` – dois elementos são iguais se têm o mesmo símbolo.
5. Getters (use `@property`) para todos os atributos.

Classe Atomo

Representa um **átomo individual** dentro de uma molécula. Cada átomo tem um identificador único global (gerado automaticamente) e uma referência ao seu elemento químico.

Atributos

- `elemento` (objeto `Elemento`)
- `id` (int) – sequencial, gerado por um contador de classe (ex.: `Atomo._proximo_id`)

Métodos obrigatórios

1. `__init__(self, elemento)` – incrementa o contador global e atribui um novo id.
2. `__str__(self)` – ex.: "H#1", "C#5" (símbolo do elemento + # + id).
3. `__repr__(self)` – pode ser igual ao `__str__`.
4. Getters para `elemento` e `id` (atributos somente leitura).

Classe Ligacao

Representa uma ligação covalente entre dois átomos individuais.

Atributos

- `atomo1, atomo2` (objetos `Atomo`)
- `ordem` (int) – 1, 2 ou 3 (simples, dupla, tripla)

Métodos obrigatórios

1. `__init__(self, atomo1, atomo2, ordem=1)` – valide:
 - `atomo1` e `atomo2` não podem ser o mesmo objeto.
 - `ordem` deve ser 1, 2 ou 3.
 - Levante `ValueError` se inválido.
2. `get_outro_atomo(self, atomo)` – retorna o outro átomo ou `None`.
3. `__str__(self)` – ex.: "H#1 - O#2 (simples)" ou "C#3 ≡ C#4 (tripla)".

Classe Molécula

Gerencia uma coleção de átomos e as ligações entre eles. A composição (quais elementos e quantidades) é **derivada automaticamente** da lista de átomos, garantindo consistência.

Atributos

- `nome (str)` – nome comum da molécula.
- `atomos (lista de Atomo)` – todos os átomos individuais.
- `ligacoes (lista de Ligacao)` – todas as ligações.

Métodos obrigatórios

1. `__init__(self, nome)` – inicializa as listas vazias.
2. `adicionar_elemento(self, elemento, quantidade=1)` – cria quantidade novos átomos do elemento e os adiciona à lista `atomos`. Cada novo átomo é uma instância de `Atomo` com ID único. Não deve criar ligações automaticamente.
3. `remover_atomo(self, atomo)` – remove um átomo específico da lista `atomos` e também remove todas as ligações em que ele participa (buscar em `ligacoes`).
4. `adicionar_ligacao(self, atomo1, atomo2, ordem=1)` – adiciona uma ligação entre os dois átomos **desde que ambos pertençam a `self.atomos`**. Verifique se a ligação já existe (na mesma ordem) para evitar duplicatas. Levante `ValueError` se algum átomo não estiver presente.
5. `calcular_composicao(self)` -> dict – percorre `self.atomos` e retorna um dicionário {Elemento: quantidade}.
6. `calcular_massa_molecular(self)` -> float – soma (`elemento.massa_atomica * quantidade`) para cada entrada do dicionário de composição.
7. `exibir_formula(self)` -> str – gera a fórmula molecular a partir do dicionário de composição. Ordem: Carbono, Hidrogênio, depois os demais em ordem alfabética do símbolo. Ex.: "H2O", "C6H12O6".
8. `obter_vizinhos(self, atomo)` -> list – retorna uma lista de tuplas (`atomo_vizinho, ordem`) para todas as ligações que contêm `atomo`.

9. `obter_grau(self, atomo) -> int` – retorna a soma das ordens das ligações que incidem sobre atomo. (Ex.: uma ligação dupla contribui com 2.)
10. `validar_valencia(self) -> bool` – para cada átomo em `self.atomos`, verifica se `self.obter_grau(atomo) <= atomo.elemento.valencia_maxima`. Se todos respeitarem, retorna `True`; caso contrário, retorna `False` e pode opcionalmente imprimir uma mensagem apontando o átomo problemático.
11. `__str__(self)` – retorna string formatada: `<nome>: <formula>, massa=<valor> u, <num_atomos> átomos, <num_ligacoes> ligações`.

Nota sobre a geração de IDs

A classe `Atomo` deve ter um atributo de classe `_proximo_id` que começa em 1. Cada vez que um novo átomo é criado, esse contador é usado e incrementado. Dessa forma, cada instância tem um ID único em toda a execução do programa.

Exemplo de uso esperado

```
1  # Criando elementos quimicos
2  H = Elemento("H", "Hidrogenio", 1, 1.008, 1)
3  O = Elemento("O", "Oxigenio", 8, 16.00, 2)
4  C = Elemento("C", "Carbono", 6, 12.011, 4)
5
6  # Molecula de agua
7  agua = Molecula("Agua")
8  agua.adicionar_elemento(H, 2)
9  agua.adicionar_elemento(O, 1)
10
11 # Recuperando os atomos (ids serao 1,2 para H e 3 para O)
12 atomos_H = [a for a in agua.atomos if a.elemento == H] # lista com 2
13             atomos
14
15 oxigenio = [a for a in agua.atomos if a.elemento == O][0]
16
17 # Adicionando ligacoes
18
19 agua.adicionar_ligacao(atomos_H[0], oxigenio, 1)
20 agua.adicionar_ligacao(atomos_H[1], oxigenio, 1)
21
22 print(agua)
23 print(f"Valencia valida? {agua.validar_valencia()}")
24 print(f"Vizinhos do O: {[ (str(v), ord) for v, ord in agua.
25                             obter_vizinhos(oxigenio)]}")
26
27 # Molecula de etanol (C2H6O) - estrutura CH3-CH2-OH
28 etanol = Molecula("Etanol")
29 etanol.adicionar_elemento(C, 2)
30 etanol.adicionar_elemento(H, 6)
31 etanol.adicionar_elemento(O, 1)
```

```

29 # Para simplificar, vamos supor que pegamos as instancias manualmente
30 # (em um sistema real seria necessario identificar cada carbono)
31 # Aqui apenas adicionariamos as ligacoes de acordo com a estrutura.

```

Listing 1: Código de demonstração

Observação: Para moléculas maiores, é comum criar métodos auxiliares que ajudem a adicionar ligações entre átomos recém-criados (ex.: usando índices ou dicionários). Fique à vontade para estender a classe.

Desafios Extras (para nota ou aprofundamento)

1. **Leitura de fórmula estrutural** – a partir de uma string como "H2O" ou "C2H5OH", crie automaticamente os átomos e as ligações básicas (poderiam ser todas simples, mas já é um avanço).
2. **Geometria molecular** – adicione coordenadas (x,y,z) à classe Atomo e implemente um método `calcular_angulo_ligacao(atomo1, atomo2, atomo3)`.
3. **Isômeros** – crie duas moléculas com a mesma fórmula molecular mas ligações diferentes e use o método `validar_valencia` para confirmar que ambas são válidas.
4. **Polaridade de ligação** – inclua eletronegatividade na classe Elemento e implemente um método `polaridade` na classe Ligacao que retorna "apolar", "polar" ou "ionica" segundo a diferença de eletronegatividade.
5. **Serialização** – implemente métodos `to_json()` e `from_json()` para salvar e carregar moléculas em arquivos.

CrITÉRIOS de Avaliação

CrITÉrio	Pontos
Classe Elemento (atributos, validações, <code>__hash__</code> , getters)	1,5
Classe Atomo (IDs automáticos, <code>__str__</code> , associação com Elemento)	1,5
Classe Ligacao (validações, <code>get_outro_atomo</code> , <code>__str__</code>)	1,5
Classe Molecula – método <code>adicionar_elemento</code> e lista <code>atomos</code>	1,0
Classe Molecula – <code>calcular_composicao</code> , <code>calcular_massa_molecular</code> , <code>exibir_formula</code>	2,0
Classe Molecula – gerenciamento de ligações (<code>adicionar_ligacao</code> , <code>obter_vizinhos</code> , <code>obter_grau</code>)	1,5
Método <code>validar_valencia</code> e integridade das estruturas	1,0
Qualidade do código (comentários, nomes, tratamento de erros)	1,0
Desafio extra (até escolha dois)	+2,0
Total máximo: 10 pontos	

Entregáveis

- Arquivo(s) .py com as classes implementadas.

- Script de demonstração (ex.: `main.py`) que cria pelo menos duas moléculas diferentes (ex.: água e metano, ou etanol), exibe suas fórmulas, massas, validação de valência e mostra os vizinhos de um átomo.
- `README.md` opcional explicando a modelagem e como executar.

Dicas importantes

- Para evitar que o contador de IDs seja reiniciado a cada instância, use um atributo de classe: `__next_id = 1` dentro de `__init__`: `self.id = Atomo.__next_id; Atomo.__next_id += 1`.
- O dicionário de composição deve ser sempre calculado a partir de `self.atomos`. Nunca o armazene como atributo mutável separado.
- Ao remover um átomo, não se esqueça de varrer a lista de ligações e remover aquelas que o envolvem (use filtro ou compreensão de lista).
- A validação de valência é um excelente teste de sanidade: se uma molécula desrespeita a valência de algum elemento, ela não pode existir na realidade.

Bom trabalho!